

Non-Graded Project: Comprehensive Application Security Assessment: From Code to Container

Learning Objective

- By the end of this activity, learners will be able to do:
Conduct end-to-end security assessments integrating multiple security disciplines and tools
 - Apply secure coding practices, threat modeling, supply chain analysis, and container security in a unified workflow
 - Synthesize security findings from different assessment phases to create prioritized risk recommendations
-

- Generate executive-level security reports that communicate technical risks in business terms
 - Demonstrate proficiency with industry-standard security tools (GitHub Codespaces, Deciduous, Syft, Grype, Trivy) in an integrated assessment
 - Develop actionable remediation strategies that address security vulnerabilities across the entire application lifecycle
-

Description

This capstone project integrates all course learning objectives through a comprehensive security assessment of Juice Shop. Students will analyze this intentionally vulnerable PHP application from multiple security perspectives - examining source code vulnerabilities, modeling potential attack paths, assessing supply chain risks, and evaluating container security posture. The project simulates the workflow of a professional security consultant conducting a holistic security review, requiring students to synthesize findings from different security disciplines into cohesive business recommendations.

Structure of the Project

Scenario

SecureWeb Solutions has been contracted by a financial services company to conduct a security assessment of their web application before migrating it to a modern cloud infrastructure. The application, based on the Juice Shop codebase, contains intentional vulnerabilities that represent common security issues found in real-world applications. As the lead security consultant, you must evaluate the application's current security posture across multiple dimensions: source code vulnerabilities, architectural attack vectors, dependency risks, and container deployment security. The client needs a comprehensive risk assessment with prioritized remediation recommendations to guide their secure modernization efforts.

Objective

Conduct a complete application security assessment of the Juice Shop application that identifies vulnerabilities across code, architecture, dependencies, and deployment infrastructure, then synthesize findings into an executive-ready security report with prioritized remediation recommendations.



Follow these steps to do this project

Step 1: Secure Code Analysis

Run `git clone https://github.com/juice-shop/juice-shop.git` (or clone [your own fork](#) of the repository)

Review the code for vulnerabilities or use a static analysis tool

Select 2 critical vulnerabilities (e.g., SQL Injection in `routes/search.ts` or XSS in user review functionality in `routes/reviews.ts`)

Alternatively, you may use a static analysis tool such as Semgrep to scan the code.

Document the vulnerable code patterns and apply secure coding principles to create fixed versions

Take screenshots comparing the vulnerable code with your secure implementations

Step 2: Application Threat Modeling

Use Deciduous to create an attack tree for Juice Shop's web application architecture

Model at least 3 primary attack paths: SQL injection leading to database compromise, XSS leading to session hijacking, and file upload leading to remote code execution

Define relevant security mitigations for each attack vector (input validation, output encoding, file type restrictions)

Set the attacker's goal as: "Gain administrative access and extract sensitive user data"

Export or screenshot the completed attack tree showing attack paths and mitigations

Step 3: Supply Chain Security Assessment

Pull the OWASP Juice Shop container image:

```
docker pull bkimminich/juice-shop:latest
```

Generate comprehensive SBOMs using Syft (should be installed from previous HOL):

```
syft bkimminich/juice-shop:latest -o json --file juice-shop-sbom.json
```

Create additional SPDX format SBOM:

```
syft bkimminich/juice-shop:latest -o spdx-json --file juice-shop-sbom-spdx.json
```

Scan the SBOM with Gype (should be installed from previous HOL):

```
gype sbom:juice-shop-sbom.json -o json --file juice-shop-  
vulnerabilities.json
```

Analyze vulnerability scan results focusing on Node.js runtime, npm packages, Angular framework dependencies, and underlying OS vulnerabilities

Document total vulnerabilities by severity and identify critical JavaScript packages requiring immediate updates

Step 4: Container Security Evaluation

Use Trivy to scan the Juice Shop container(should be installed from previous HOL):

```
trivy image bkimminich/juice-shop:latest
```

Generate detailed JSON report:

```
trivy image --format json --output juice-shop-trivy-report.json  
bkimminich/juice-shop:latest
```


Create HTML report for stakeholders:

```
trivy image --format template --template html.tpl --output juice-shop-security-report.html bkimminich/juice-shop:latest
```

Scan for misconfigurations and secrets:

```
trivy image --scanners vuln,misconfig,secret bkimminich/juice-shop:latest
```

Document critical container security findings including Node.js base image vulnerabilities, exposed configuration files, and potential hardcoded secrets

Step 5: Risk Assessment and Prioritization

Synthesize findings from code analysis, threat modeling, supply chain scanning, and container security assessment

Apply risk scoring considering CVSS scores, exploitability (Juice Shop contains intentional vulnerabilities), and e-commerce business impact

Identify the top 5 security issues requiring immediate attention across all assessment phases

Map security findings to business impacts: customer data breach, payment fraud, compliance violations (PCI-DSS), reputational damage

Create a risk matrix prioritizing vulnerabilities that affect customer trust and financial transactions

Step 6: Compile the Executive Security Report

Organize all findings, evidence, and recommendations into the structured report format

Create executive summary highlighting critical risks to e-commerce operations and customer data protection

Ensure all screenshots, scan outputs, code examples, and threat models are properly documented

Verify professional presentation quality suitable for executive stakeholders in retail/e-commerce industry

Organize information following this structure for peer review

Title Page: Project title, your name, date, application security assessment.

Project Requirement 1: Code Security Assessment

Document specific vulnerabilities analyzed (SQL injection, XSS, file upload, etc.)

Provide before/after code comparisons showing secure coding implementations

Explain security principles applied: input validation, parameterized queries, output encoding, access controls

Include screenshots from GitHub Codespaces showing vulnerable code analysis and remediation examples

Project Requirement 2: Threat Model and Attack Path Analysis

Present the completed attack tree created with Deciduous

Describe primary attack vectors: injection attacks, cross-site scripting, authentication bypass, file upload exploitation

Document security mitigations for each threat path (WAF rules, input validation, session management)

Explain how threat modeling reveals the interconnected nature of the vulnerabilities

Project Requirement 3: Supply Chain and Dependency Risk Assessment

Provide vulnerability statistics from the container SBOM analysis using Syft and Gype

Highlight critical PHP, Apache, MySQL, and system package vulnerabilities

Document total vulnerable components by severity
(Critical/High/Medium/Low)

Include specific CVEs and packages that pose the highest risk to the application stack

Project Requirement 4: Container Security Posture Analysis

Present comprehensive findings from Trivy container security scan

Document base image vulnerabilities, configuration issues, and security misconfigurations

Include generated JSON and HTML security reports demonstrating thorough container analysis

Compare container scan results with supply chain analysis to identify overlapping risks

Recommendations: Integrated Security Strategy

Provide one comprehensive, actionable recommendation addressing the most critical finding from your integrated assessment

Prioritize immediate remediation of exploitable vulnerabilities (SQL injection, XSS) alongside infrastructure security improvements

Include specific implementation guidance: secure coding practices, container hardening, dependency management, and security monitoring

Address both tactical fixes (patch critical CVEs, implement input validation) and strategic improvements (security architecture, DevSecOps integration)

Instructions for Documenting and Sharing the Project for Peer Review



Document the Project

Use word processing software to create your comprehensive security assessment report

Insert all screenshots, scan outputs, attack trees, and vulnerability data directly into the document

Ensure all sections include specific references to vulnerabilities and container security findings

Create an executive summary that translates technical findings into business risks and modernization requirements

Proofread and edit your report for clarity, accuracy, and professional presentation quality

Share the Project



Save your comprehensive security assessment report in PDF format

Create a supplementary document containing all raw scan outputs, SBOMs, Trivy reports, and security analysis as appendices

Upload your documents to the "Peer Review" area in the course shell

Provide a brief description: "Comprehensive Juice Shop application security assessment identifying critical web application vulnerabilities, container security risks, and supply chain threats with integrated remediation strategy"

Peer Review



Review the security assessments submitted by your peers, evaluating their integration of multiple security assessment methodologies

Assess whether their threat models accurately represent realistic attack scenarios against web applications

- 
- Evaluate the thoroughness of their container and supply chain security analysis
 - Check if their code-level vulnerability analysis demonstrates understanding of secure coding principles
 - Provide constructive feedback on their risk prioritization methodology and remediation recommendations
 - Comment on how effectively they synthesized findings from different tools into cohesive business recommendations
-